

Prerequisites

1. Java

- Java Version: OpenJDK 17 (<https://openjdk.org/>)

2. Database

- SQL Server 2025 Express Or [Standard Developer edition](#)
 - Used as the relational database for the application
- SQL Server Management Studio (SSMS)
 - Used to manage database schema, tables, and data

3. IDE (Integrated Development Environment)

- IntelliJ IDEA

4. Build Tools

- **Maven** (for WAR/JAR build)

5. Postman – For testing REST APIs

--Second Session Tasks--

. Assignment:

Library Management System using Spring Boot 3

Objective

Develop **System** using **Spring Boot (latest version)** that demonstrates **layered architecture, JPA database connectivity, logging, global exception handling, SQL Server integration, and Swagger API documentation.**

Technical Requirements

- Spring Boot (Use version 3)
- Java 17 or above

- Maven
- Spring Data JPA
- SQL Server database
- SLF4J / Logback logging
- RESTful API design
- **The following features must be implemented in the project:**
 - a. JPQL Queries
 - b. Native SQL Queries
 - c. JPA Criteria API
 - d. DTO Mapping
 - e. Proper API Validation
 - f. Pagination and Sorting
 - g. Entity relation one to one / one to many/many to many

Library Management System – Entity Details

1. Library

- Represents the library information.
 - Stores basic details such as library name and location.
 - One library can have multiple books.
-

2. Book

- Represents a book available in the library.
 - Contains details like title, ISBN, and publication year.
 - Each book belongs to one library.
 - A book can have multiple authors.
-

3. Author

- Represents the author of a book.
- Stores author name and biography (optional).

- One author can write multiple books.
-

4. Member

- Represents a registered library member.
 - Contains member details like name, email, and membership date.
 - A member can borrow multiple books over time.
-

5. BorrowRecord

- Represents a book borrowing transaction.
- Stores borrow date, return date, and borrowing status.
- Links a member to a book.
- Used to maintain borrowing history.

6. MemberProfile

MemberProfile stores the additional personal information of a library member that is not part of the core Member entity.

Mapping Type Entities

One-to-Many Library → Book

Many-to-One Book → Library

Many-to-Many Book ↔ Author

One-to-Many Member → BorrowRecord

One to One Member to MemberProfile

Many -to-One BorrowRecord ↔ Book

Note:

Spring Boot Gradle Project – REST API with Layered Architecture

(Using OOPS Concepts)

Create classes using:

- @Component
- @Service
- @Repository

Use @Autowired in:

- Field Injection
- Constructor Injection

Mandatory OOPS Concepts to Follow

The implementation **must demonstrate** the following OOPS concepts:

1. **Encapsulation**
2. **Abstraction**
3. **Inheritance**
4. **Polymorphism**